



EXVUL

SMART CONTRACT **AUDIT REPORT**

DHF Finance Smart Contract

MARCH 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Merkle Claims Can Exceed Configured Total Reward	9
3.2 Node payment asset and liquidity base asset inconsistency causes sale denial and treasury asset misconsumption	11
3.3 bonusTotalSupply Miscalculates Eligible Holder Supply	13
3.4 Manipulable Spot Pricing And Zero Slippage Floors Expose User And Treasury Trades To MEV Extraction	15
3.5 Boundary Condition Error in Minimum Holding Validation	17
3.6 Reward denominator keyed by fixed node ID can over-distribute to all same-type buyers	19
3.7 nextMerkleUpdateTime Is Ineffective and Leaves Merkle Root Updates Unconstrained . .	21
3.8 Root Error Code Semantics Are Inaccurate	23
3.9 DAO rate annotation markup error	24
4. CONCLUSION	25
5. APPENDIX	26
5.1 Basic Coding Assessment	26
5.1.1 Apply Verification Control	26
5.1.2 Authorization Access Control	26
5.1.3 Forged Transfer Vulnerability	26
5.1.4 Transaction Rollback Attack	27
5.1.5 Transaction Block Stuffing Attack	27
5.1.6 Soft Fail Attack Assessment	27
5.1.7 Hard Fail Attack Assessment	28
5.1.8 Abnormal Memo Assessment	28
5.1.9 Abnormal Resource Consumption	28
5.1.10 Random Number Security	29
5.2 Advanced Code Scrutiny	29
5.2.1 Cryptography Security	29

5.2.2 Account Permission Control	29
5.2.3 Malicious Code Behavior	30
5.2.4 Sensitive Information Disclosure	30
5.2.5 System API	30
6. DISCLAIMER	31
7. REFERENCES	32
8. About Exvul Security	33

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **DHF Finance** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
DHF Finance	31/03/2026 - 01/04/2026	Solidity

Repository

<https://github.com/DHFfinance/dhf>

Commit Hash

1683e0206a8ec86b469cde80981bb3373a0da6d4

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	3
LOW	4
INFO	2



2.3 Key Findings

Severity	Findings Title	Status
MEDIUM	Merkle Claims Can Exceed Configured Total Reward	Fixed
MEDIUM	Node payment asset and liquidity base asset inconsistency causes sale denial and treasury asset misconsumption	Fixed
MEDIUM	bonusTotalSupply Miscalculates Eligible Holder Supply	Fixed
LOW	Manipulable Spot Pricing And Zero Slippage Floors Expose User And Treasury Trades To MEV Extraction	Fixed
LOW	Boundary Condition Error in Minimum Holding Validation	Fixed
LOW	Reward denominator keyed by fixed node ID can over-distribute to all same-type buyers	Fixed
LOW	nextMerkleUpdateTime Is Ineffective and Leaves Merkle Root Updates Unconstrained	Fixed
INFO	Root Error Code Semantics Are Inaccurate	Fixed
INFO	DAO rate annotation markup error	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Merkle Claims Can Exceed Configured Total Reward

SEVERITY:

MEDIUM

STATUS:

Fixed

PATH:

contracts/BonusPool.sol

DESCRIPTION:

The `claim()` function calculates the user's deserved reward as `totalAmount_ - claimedAmt_`, but it does not check whether `merkleData.merkleTotalClaimed + reward_` exceeds `merkleData.merkleTotalReward`. A valid proof with an oversized cumulative amount can therefore push total claims beyond the reward budget configured for the active Merkle round.

```
function claim(
    uint256 totalAmount_,
    bytes32[] calldata merkleProof_
) external {
    // ...
    uint256 reward_ = totalAmount_ - claimedAmt_;

    merkleClaimedAmounts[account_] = totalAmount_;
    merkleData.merkleTotalClaimed += reward_;
    userClaimedState[account_][merkleVersion_] = true;

    IERC20Token(XTOKEN).safeTransfer(account_, reward_);

    emit EventClaim(account_, reward_, totalAmount_);
}
```

IMPACT:

The Merkle claim path can over-distribute reward inventory and exceed project expectations, which creates accounting drift and can starve later eligible users of the tokens reserved for their round.

RECOMMENDATIONS:

Validate the updated cumulative claimed amount against `merkleData.merkleTotalReward` before mutating claim state or transferring tokens.

```
function claim(
    uint256 totalAmount_,
    bytes32[] calldata merkleProof_
) external {
    // ...
    uint256 reward_ = totalAmount_ - claimedAmt_;
+   if (merkleData.merkleTotalClaimed + reward_ >
merkleData.merkleTotalReward) revert ErrorClaimExceed();

    merkleClaimedAmounts[account_] = totalAmount_;
    merkleData.merkleTotalClaimed += reward_;
    userClaimedState[account_][merkleVersion_] = true;

    IERC20Token(XTOKEN).safeTransfer(account_, reward_);

    emit EventClaim(account_, reward_, totalAmount_);
}
```

3.2 Node payment asset and liquidity base asset inconsistency causes sale denial and treasury asset misconsumption

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

contracts/NFTPresell.sol

DESCRIPTION:

`nftPresell()` allows managers to configure an arbitrary `payToken`, but the downstream liquidity path is hard-wired to `USDT` and `XTOKEN`. This creates a broken accounting path where user-paid non-USDT assets are disconnected from the market-making asset that the internal swap and liquidity operations actually consume.

```
// user pays configurable token
function nftPresell(uint256 nodeId_) external
updateRewards(msg.sender) {
    // ...
    IERC20Token(config_.payToken).safeTransferFrom(msg.sender,
address(this), config_.payAmount);
    uint256 liquidity_ = _addLiquidity(config_.payAmount);
    // ...
}

// internal path is fixed to USDT
function _addLiquidity(uint256 tokenAmount_) internal
returns(uint256) {
    uint256 swappedXAmount_ = _swapUSDToX(tokenAmount_/2,
address(this));
    (, uint256 costX_, uint256 liquidity_) =
IRouter(SWAPROUTER).addLiquidity(
    USDT,
    XTOKEN,
    false,
    tokenAmount_/2,
    swappedXAmount_,
    0,
    0,
```

```
        BLACK_HOLE,
        block.timestamp
    );
    if (costX_ < swappedXAmount_) {
        IERC20Token(XTOKEN).safeTransfer(JLP_USDT_X, swappedXAmount_
- costX_);
        IPool(JLP_USDT_X).sync();
    }
    emit EventAddLiquidity(tokenAmount_, swappedXAmount_, liquidity_);
    return liquidity_;
}
```

IMPACT:

If a node is configured with a non-USDT payment token, purchases can revert when the contract lacks USDT, fully disabling that sale path. If the contract already holds USDT inventory, treasury USDT can be consumed while user-paid non-USDT assets accumulate as stranded balances.

RECOMMENDATIONS:

Enforce a single payment base such as USDT at configuration time, or refactor the liquidity path so it consumes `config_.payToken` consistently.

```
+   error ErrorPayTokenNotUSDT();
    function setNodeConfigs(NodeConfigParam[] calldata nodeConfigs_)
    external onlyManager {
        uint256 len_ = nodeConfigs_.length;
        for(uint256 i=0;i<len_;i++) {
+           if (nodeConfigs_[i].payToken != USDT) revert
ErrorPayTokenNotUSDT();
            nodeConfigs[nodeConfigs_[i].nodeId] = NodeConfig(
                nodeConfigs_[i].payToken,
                nodeConfigs_[i].payAmount,
                nodeConfigs_[i].nType,
                nodeConfigs_[i].maxLimit
            );
        }
    }
}
```

3.3 bonusTotalSupply Miscalculates Eligible Holder Supply

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

contracts/TokenX.sol

DESCRIPTION:

`TokenX.sol` explicitly marks several addresses as ignored for bonus distribution, but `bonusTotalSupply()` excludes only a subset of them. The balances of `address(this)`, `stakePool_`, and `IPool(jLpXUSDt).poolFees()` still remain in the reward denominator even though those balances are not meant to participate in hold-bonus distribution.

```
constructor(
    string memory name_,
    string memory symbol_,
    address bonusPool_,
    address stakePool_
) ERC20(name_, symbol_) ERC20Permit(name_) {
    // ...

    lastDay = block.timestamp / 86400;
    ignoreList[LP_TOKEN_BNB] = true;
    ignoreList[bonusPool_] = true;
    ignoreList[stakePool_] = true;
    ignoreList[address(this)] = true;
    ignoreList[BLACK_HOLE] = true;

    _mint(msg.sender, 2100 * 1e8 * 1e18);
}

function setJLP() external onlyTokenManager {
    address lp_ = JSWAPFACTORY.createPool(address(this), USDT, false);
    jLpXUSDt = lp_;
    ignoreList[lp_] = true;
    ignoreList[IPool(lp_).poolFees()] = true;
}

function bonusTotalSupply() public view returns(uint256) {
    return totalSupply()
```

```
        - this.balanceOf(jLpXUSDt)
        - this.balanceOf(LP_TOKEN_BNB)
        - this.balanceOf(BLACK_HOLE)
        - this.balanceOf(BONUSPOOL);
    }
    function _toBonus(uint256 amount_) internal {
        uint256 totalSupply_ = bonusTotalSupply();
        if (totalSupply_ == 0) {
            return;
        }
        uint256 ratio_ = (amount_ * 1e18) / totalSupply_;
        uint256 index_ = index;
        if (ratio_ > 0) {
            index_ += ratio_;
            index = index_;
        }
        emit EventToHoldBonus(amount_, ratio_, index_, totalSupply_);
    }
}
```

IMPACT:

Eligible holders receive less hold bonus than intended because non-eligible balances inflate the denominator and dilute the reward ratio.

RECOMMENDATIONS:

Exclude every ignored balance from `bonusTotalSupply()` so the distribution ratio is calculated only across addresses that are actually eligible.

```
function bonusTotalSupply() public view returns(uint256) {
    return totalSupply()
        - this.balanceOf(jLpXUSDt)
        - this.balanceOf(LP_TOKEN_BNB)
        - this.balanceOf(BLACK_HOLE)
+       - this.balanceOf(STAKEPOOL)
+       - this.balanceOf(address(this))
+       - this.balanceOf(IPool(jLpXUSDt).poolFees())
        - this.balanceOf(BONUSPOOL);
}
```

3.4 Manipulable Spot Pricing And Zero Slippage Floors Expose User And Treasury Trades To MEV Extraction

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/StakePool.sol

contracts/NFTPresell.sol

contracts/TokenX.sol

DESCRIPTION:

`StakePool` computes `minAmount_` from the same `JLP_USDT_X` reserves that it is about to trade against, while related flows in `NFTPresell` and `TokenX` pass 0 as the swap output floor. The same execution paths also add liquidity with zero minimum amounts, which leaves users and treasury-managed trades open to MEV-driven price manipulation.

```
function _swapUSDToX(uint256 amount_, address recipient_) internal
    returns (uint256){
    IRouter.Route[] memory route_ = new IRouter.Route[](1);
    route_[0] = _getRouteFromToken(USDT, XTOKEN, false);
    uint256 minAmount_ = _getTokenPrice(JLP_USDT_X, USDT, amount_) * 97 /
    100;
    uint256[] memory amounts_ =
    IRouter(SWAPROUTER).swapExactTokensForTokens(amount_, minAmount_,
    route_, recipient_, block.timestamp);
    return amounts_[amounts_.length - 1];
}

function _swapUSDToX(uint256 amount_, address recipient_) internal
    returns (uint256){
    IRouter.Route[] memory route_ = new IRouter.Route[](1);
    route_[0] = _getRouteFromToken(USDT, XTOKEN, false);
    uint256[] memory amounts_ =
    IRouter(SWAPROUTER).swapExactTokensForTokens(amount_, 0, route_,
    recipient_, block.timestamp);
    return amounts_[amounts_.length - 1];
}
```

```
function _swapToNative(
    uint256 amount_,
    address recipient_
) internal returns (uint256) {
    address[] memory path_ = new address[](2);
    path_[0] = address(this);
    path_[1] = WBNB;

    uint256 valueBefore_ = recipient_.balance;
    SWAPROUTER.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amount_,
        0,
        path_,
        recipient_,
        block.timestamp
    );
    return recipient_.balance - valueBefore_;
}
```

IMPACT:

Attackers can sandwich `stake()`, `sellToken()`, `nftPresell()`, and fee-liquidation flows, forcing materially worse execution and extracting value from users or protocol fee inventory.

RECOMMENDATIONS:

Derive minimum outputs from a fresh quote and enforce non-zero slippage bounds across swaps and liquidity additions.

```
+ uint256 quotedOut_ = _getTokenPrice(JLP_USDT_X, USDT, amount_);
- uint256 minAmount_ = _getTokenPrice(JLP_USDT_X, USDT, amount_) * 97 /
  100;
+ uint256 minAmount_ = quotedOut_ * 99 / 100;

+ uint256 minAmountOut_ = SWAPROUTER.getAmountsOut(amount_, path_)[1] *
  99 / 100;
-      0,
+      minAmountOut_,
```


3.5 Boundary Condition Error in Minimum Holding Validation

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/BonusPool.sol

DESCRIPTION:

`claimHold()` checks the user's balance with `<= CLAIM_MIN_HOLD`, so holders with exactly the documented minimum amount are treated as ineligible even though they satisfy the threshold.

```
function claimHold() external {
    address user_ = msg.sender;
    if (IERC20TokenX(XTOKEN).balanceOf(user_) <= CLAIM_MIN_HOLD)
        revert ErrorHoldTooLittle();

    // ...

    emit EventClaimHoldBonus(user_, claimables_, index_);
}
```

IMPACT:

Even users who meet the minimum holding requirement can be blocked from claiming rewards, which directly affects normal user experience.

RECOMMENDATIONS:

Use a strict less-than comparison so the minimum balance itself remains eligible.

```
function claimHold() external {
    address user_ = msg.sender;
-   if (IERC20TokenX(XTOKEN).balanceOf(user_) <= CLAIM_MIN_HOLD)
+   if (IERC20TokenX(XTOKEN).balanceOf(user_) < CLAIM_MIN_HOLD)
        revert ErrorHoldTooLittle();
}
```

```
        // ...  
  
        emit EventClaimHoldBonus(user_, claimables_, index_);  
    }
```

3.6 Reward denominator keyed by fixed node ID can over-distribute to all same-type buyers

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/NFTPresell.sol

DESCRIPTION:

`notifySwapFeeRewards()` and related claim logic key their denominator to fixed node IDs while user eligibility is checked only by `nType`. If multiple node IDs are configured under the same reward-bearing type, all of those users can claim against a pool whose denominator counted only one node population.

```
function notifySwapFeeRewards(uint256 smallAmount_, uint256
daoAmount_) external onlyBonusPool {
    if (smallAmount_ > 0) {
        uint256 rewardPerStoredNew_ =
swapFeeSmallRewardData.rewardPerStored;
        uint256 nodeNum_ = nodeCurNums[_SMALLNODEID];
        if (nodeNum_ > 0) {
            rewardPerStoredNew_ += smallAmount_ / nodeNum_;
            swapFeeSmallRewardData.rewardPerStored =
rewardPerStoredNew_;
        }
        // ...
    }

    function swapFeeSmallEarnedNew(address user_) public view returns
(uint256) {
        if (purchaseList[user][_SMALLTYPE].amount == 0) {
            return 0;
        }
        return swapFeeSmallRewardData.rewardPerStored -
swapFeeSmallRewardData.userRewardPerStored[user_];
    }
}
```

IMPACT:

If multiple node IDs share the same reward-bearing type, total claimable rewards can exceed the amount allocated for that round, allowing early claimers to drain [BonusPool](#) while later claimers fail.

RECOMMENDATIONS:

Enforce a one-to-one mapping between reward-bearing [nType](#) values and node IDs.

```
+   error ErrorDuplicatedRewardType();
    function setNodeConfigs(NodeConfigParam[] calldata nodeConfigs_)
    external onlyManager {
        uint256 len_ = nodeConfigs_.length;
        for(uint256 i=0;i<len_;i++) {
+           if (nodeConfigs_[i].nType == _SMALLTYPE &&
nodeConfigs_[i].nodeId != _SMALLNODEID) revert
ErrorDuplicatedRewardType();
+           if (nodeConfigs_[i].nType == _DAOTYPE &&
nodeConfigs_[i].nodeId != _DAONODEID) revert
ErrorDuplicatedRewardType();
                nodeConfigs[nodeConfigs_[i].nodeId] = NodeConfig(
                    nodeConfigs_[i].payToken,
                    nodeConfigs_[i].payAmount,
                    nodeConfigs_[i].nType,
                    nodeConfigs_[i].maxLimit
                );
        }
    }
```

3.7 nextMerkleUpdateTime Is Ineffective and Leaves Merkle Root Updates Unconstrained

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/BonusPool.sol

DESCRIPTION:

`nextMerkleUpdateTime` is configurable, but `updateMerkleRoot()` never enforces it. The root manager can therefore update the Merkle root at any time despite the presence of a timing-control variable.

```
function updateMerkleRoot(bytes32 merkleRoot_, uint256
merkleVersion_, uint256 newTotalReward_) external onlyRootManager {
    if (merkleRoot_ == "") revert ErrorMerkleRootEmpty();

    if (newTotalReward_ < merkleData.merkleTotalReward) revert
ErrorRewardAmount();
    if (merkleVersion_ <= merkleData.merkleVersion) revert
ErrorVersionNumber();

    merkleData.merkleRoot = merkleRoot_;
    uint256 addTotalReward_ = newTotalReward_ -
merkleData.merkleTotalReward;
    merkleData.merkleTotalReward = newTotalReward_;
    merkleData.merkleVersion = merkleVersion_;

    emit EventUpdateMerkleRoot(merkleRoot_, merkleVersion_,
newTotalReward_, addTotalReward_);
}

function setNextMerkleUpdateTime(uint256 time_) external onlyManager {
    merkleData.nextMerkleUpdateTime = time_;
}
```

IMPACT:

The intended timing restriction is ineffective, so Merkle update frequency remains fully unconstrained.

RECOMMENDATIONS:

Check `nextMerkleUpdateTime` inside `updateMerkleRoot()` and revert when the update is attempted before the configured time.

```
function updateMerkleRoot(bytes32 merkleRoot_, uint256
merkleVersion_, uint256 newTotalReward_) external onlyRootManager {
+   if (block.timestamp < merkleData.nextMerkleUpdateTime) revert
ErrorUpdateTooEarly();
    if (merkleRoot_ == "") revert ErrorMerkleRootEmpty();

    if (newTotalReward_ < merkleData.merkleTotalReward) revert
ErrorRewardAmount();
    if (merkleVersion_ <= merkleData.merkleVersion) revert
ErrorVersionNumber();

    merkleData.merkleRoot = merkleRoot_;
    uint256 addTotalReward_ = newTotalReward_ -
merkleData.merkleTotalReward;
    merkleData.merkleTotalReward = newTotalReward_;
    merkleData.merkleVersion = merkleVersion_;

    emit EventUpdateMerkleRoot(merkleRoot_, merkleVersion_,
newTotalReward_, addTotalReward_);
}
```

3.8 Root Error Code Semantics Are Inaccurate

SEVERITY:

INFO

STATUS:

Fixed

PATH:

contracts/BonusPool.sol

DESCRIPTION:

`claim()` returns `ErrorRewardType()` when `merkleData.merkleRoot` is empty. The actual failure is not an invalid reward type, but a missing or unset Merkle root, so the current error name is semantically inaccurate.

```
function claim(
    uint256 totalAmount_,
    bytes32[] calldata merkleProof_
) external {
    bytes32 root_ = merkleData.merkleRoot;
    if (root_ == "") revert ErrorRewardType();

    // ...
}
```

RECOMMENDATIONS:

Replace `ErrorRewardType()` with a dedicated error that accurately describes an empty Merkle root.

```
function claim(
    uint256 totalAmount_,
    bytes32[] calldata merkleProof_
) external {
    bytes32 root_ = merkleData.merkleRoot;
-   if (root_ == "") revert ErrorRewardType();
+   if (root_ == "") revert ErrorMerkleRootEmpty();

    // ...
}
```

3.9 DAO rate annotation markup error

SEVERITY:

INFO

STATUS:

Fixed

PATH:

contracts/BonusPool.sol

DESCRIPTION:

The inline comment for `daoAmount_` in `allotBonus()` states `0.3%`, but the implemented expression `amount_ * 5 / 28` corresponds to `0.5%` of the original `3%` transaction fee after deducting the destroyed portion. The logic is correct, but the annotation is misleading.

```
-uint256 daoAmount_ = amount_ * 5 / 28; // 0.3%  
+uint256 daoAmount_ = amount_ * 5 / 28; // 0.5%
```


4. CONCLUSION

In this audit, we thoroughly analyzed **DHF Finance** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec